

Introduction to Machine Learning for Text Analysis with Python

Day 1 - Monday Morning

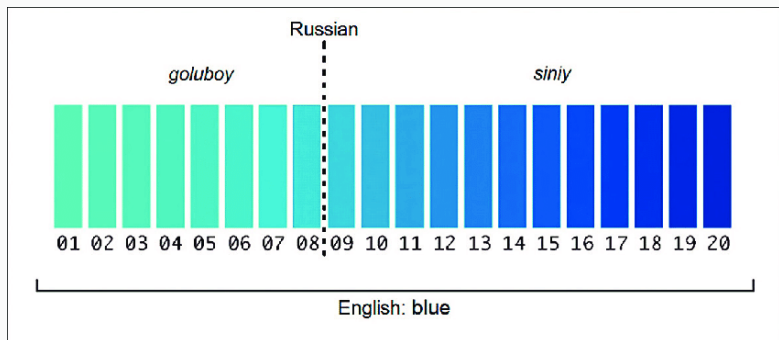
»Introduction«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 14, 2026

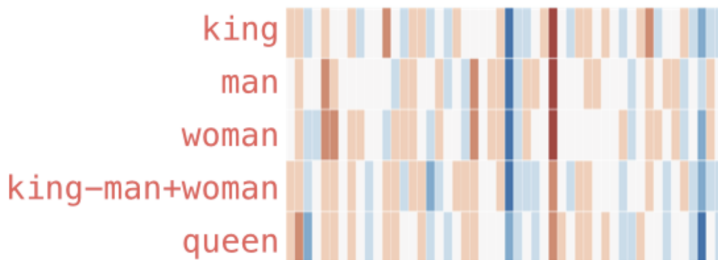
Can you see the difference?



The limits of my language mean the limits of my world - Ludwig Wittgenstein

Teaching Language to Computers

king - man + woman $\approx$ queen



Words and signs are not identical with what they signify, and only acquire meaning through their differences from other words and signs; meaning arises from the differentiation of words¹

All course materials can be found at...

<https://github.com/Rptkiddle/gesis-machine-learning-2026>

We will update these daily: 'git fetch + git pull'

Introducing. . .

Introducing. . .

. . . the people

Introducing...
...the people

Introducing. . .



Rupert Kiddle

PostDoc, PhD Candidate (RM SocSci, BSc PolSci,
BSc NeuroSci)

- Vrije University Amsterdam, Communication
- Works with: Generative Agent Based Modelling (with LLMs), Information Systems in (Social) media (recommendation, retrieval), NLP, complex systems theories / problems.

r.t.kiddle@vu.nl

Introducing. . .

Sjoerd Stolwijk

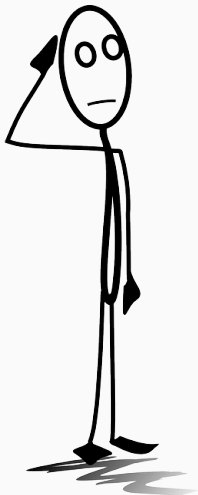
Senior researcher Utrecht School of Governance
(PhD PolCom, MA PolSci)



- Former postdoc in computational communication, University of Amsterdam
- Current project studies political decision-making under uncertainty using surveys and experiments
- Interested in biases, both in text annotation and political decision-making

s.b.stolwijk@uu.nl

Introducing...



Your name?

Your background?

Your reason to follow this course?

Your experience with python?

Do you have a dataset you are working on?

Today

- Morning
 - Overview of the course
 - Housekeeping (AI tools, good practice)
- Afternoon
 - Supervised versus Unsupervised ML
 - From text to features -> what does a computer see when it reads a text?

Course overview

The Course in a Nutshell

- Day 1** Representing text as numbers: from Dictionaries to Contextual Embeddings
- Day 2** Unsupervised Machine Learning (UML) with BERTopic
- Day 3** Supervised Machine Learning (SML) with BERT fine-tuning
- Day 4** Best Practices for SML + Project Day 1
- Day 5** Navigating the Model Ecosystem, Critical Evaluation, (and Generative AI?) + Project Day 2

Don't be afraid of the forest

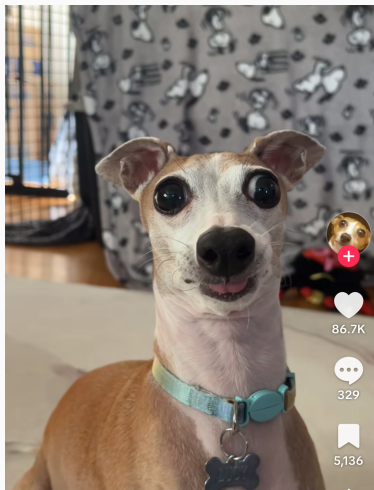
This is a beginner course

This is a challenging area to begin in

But there is a lot of 'noise' in ML/NLP.

Working through the foundations helps a lot.

This course can feel like...



What is machine learning for text analysis

- Machine Learning is a broad term
- From counting words to generative AI
- We will introduce the basics: focus on understanding the logic

What is machine learning for text analysis

- Machine Learning is a broad term
- From counting words to generative AI
- We will introduce the basics: focus on understanding the logic

What is machine learning for text analysis

- Machine Learning is a broad term
- From counting words to generative AI
- We will introduce the basics: focus on understanding the logic

Sixty years of getting closer to natural language

How programmers have printed *hello world*:

1950s – Assembly	<pre>mov dx, offset msg mov ah, 09h int 21h</pre>
1970s – C	<pre>printf("hello world\n");</pre>
1990s – Python	<pre>print('hello world')</pre>
Today – AI assistant	<pre>"print hello world"</pre>

Every step moves the interface closer to natural language.

Sixty years of getting closer to natural language

How programmers have printed *hello world*:

1950s – Assembly	<code>mov dx, offset msg</code> <code>mov ah, 09h</code> <code>int 21h</code>
1970s – C	<code>printf("hello world\n");</code>
1990s – Python	<code>print('hello world')</code>
Today – AI assistant	“print hello world”

Every step moves the interface closer to natural language.

The practice of programming has changed

- Python was designed to read almost like natural language, to make programming more accessible.
- AI assistants close this loop: you say what you want -> the AI writes the Python (or R, Rust, etc.).
- AI writes routine code faster than we do, and often with fewer mistakes.
- So the key skill has shifted: **reading** and **understanding** code now matters more than writing it by hand.

The practice of programming has changed

- Python was designed to read almost like natural language, to make programming more accessible.
- AI assistants close this loop: you say what you want -> the AI writes the Python (or R, Rust, etc.).
- AI writes routine code faster than we do, and often with fewer mistakes.
- So the key skill has shifted: **reading** and **understanding** code now matters more than writing it by hand.

The practice of programming has changed

- Python was designed to read almost like natural language, to make programming more accessible.
- AI assistants close this loop: you say what you want -> the AI writes the Python (or R, Rust, etc.).
- AI writes routine code faster than we do, and often with fewer mistakes.
- So the key skill has shifted: **reading and understanding** code now matters more than writing it by hand.

The practice of programming has changed

- Python was designed to read almost like natural language, to make programming more accessible.
- AI assistants close this loop: you say what you want -> the AI writes the Python (or R, Rust, etc.).
- AI writes routine code faster than we do, and often with fewer mistakes.
- So the key skill has shifted: **reading** and **understanding** code now matters more than writing it by hand.

Our advice for using AI this week

- Use **chat** or **in-line suggestions**. Write a comment, let the AI suggest the code. You read every line, so you learn.
- Avoid **agent** modes for now. The AI does many steps at once, and often hides decisions from you.
- Read the code before you run it. Even if you can only understand 80% of it, the other 20% will come later.

Our advice for using AI this week

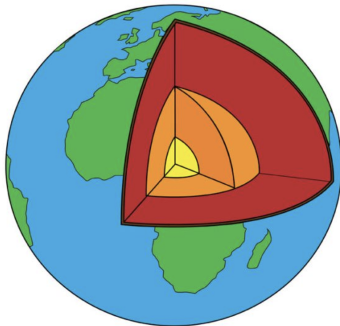
- Use **chat** or **in-line suggestions**. Write a comment, let the AI suggest the code. You read every line, so you learn.
- Avoid **agent** modes for now. The AI does many steps at once, and often hides decisions from you.
- Read the code before you run it. Even if you can only understand 80% of it, the other 20% will come later.

Our advice for using AI this week

- Use **chat** or **in-line suggestions**. Write a comment, let the AI suggest the code. You read every line, so you learn.
- Avoid **agent** modes for now. The AI does many steps at once, and often hides decisions from you.
- Read the code before you run it. Even if you can only understand 80% of it, the other 20% will come later.

Two ways of seeing this course:

1. As a 'slice' through the world of ML/NLP



The 'Core' - Text as Data

How can we turn words into numbers?

King – Man + Woman = Queen

The 'Mantle' - Paradigms

*From 'lonely symbols' (**Bag-of-Words**)*

*... to static meaning (**Word2Vec**)*

*... to contextual meaning (**Transformers**)*

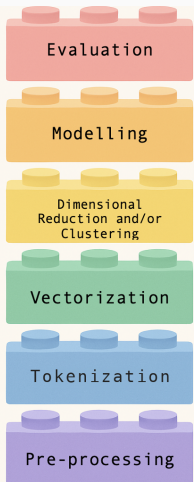
The 'Crust' - Methods / Models

The Old World vs. The New World

(pre-trained models, transfer learning)

...more nuance, but also more complexity

2. As a starter toolkit



how to assess the output of your model

how to train a model to describe (UML)
or to predict (SML)

how to group features *or* cases

how to turn text → numbers

how to split text into tokens

how to tidy up text data

What to expect

- Learn how a machine looks at text
- Learn how to implement basic applications of state-of-the-art machine learning models
- We will not go into generative AI

Housekeeping

Housekeeping

A good workflow

A good workflow

Housekeeping

Clean, high-quality code

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Start with a small, simple dataset and a simple analysis
- Gradually increase the complexity of the data and the analysis
- Use the results of the small-scale analysis to inform the larger-scale analysis

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Take your plan and solve *one* problem at a time (e.g., parsing a review page; or getting the URLs of all review pages)
- Test each step in your code with a small test sample

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Take your plan and solve *one* problem at a time (e.g., parsing a review page; or getting the URLs of all review pages)
- Test each step in your code with a small test sample

Develop components separately

If you copy-paste code your own code, you are doing something wrong

- Use loops!
- If something takes more than a couple of lines, write a function!

Develop components separately

If you copy-paste code your own code, you are doing something wrong

- Use loops!
- If something takes more than a couple of lines, write a function!

Make it robust

You cannot foresee every possible problem.

Most important: Make sure your program does not fail and loose all data just because something goes wrong at case 997/1000.

- Use an environment
- Use version control (GitHub)
- Document, document, document -> you won't remember what you did many commits down the line
- Use try/except to explicitly tell the program how to handle errors
- Write data to files (or database) in between

Dealing with versions

- You import packages per notebook
- Packages might conflict with and depend on each other -> for exercises, use our stated versions
- We will use Python 3.12. Generally a good idea to use a recent version, but not the very latest.

Your experiences

- What is your experience with Python?
- How did everyone go with setup and introductory notebooks?

This afternoon

Getting started

- Supervised versus Unsupervised ML
- From text to features -> what does a computer see when it reads a text?